Individual Report

MiniOS Alpha | Operating Systems Final Project

Student Name:

Group: Marquez and Friends

Assigned Role: Process Management

What did I contribute to the project?

I built the process table in processes.py. It simulates processes as records with a PID, a name, a status and a memory cost, and lets the user start, list, pause, resume and terminate them. I worked with Student 4 so that starting a process reserves memory and killing it gives that memory back.

What features did I personally develop?

- A Process class that holds the PID, name, status, memory used and start time
- A ProcessManager with a process table and a PID counter
- process, run and kill (the required commands)
- suspend and resume (extra commands) to switch between RUNNING and SUSPENDED
- A fixed list of programs that can be started: calculator, editor, notepad, browser, terminal and clock
- kill_all, which cleans up every process on exit and logout

What problems did I encounter?

- A PID should never point to two different processes during one session.
- Wrong input such as kill 999 (no such process) or kill abc (not a number).
- Processes could be left running after exit or logout and keep memory reserved.
- Starting a process when the simulated memory is almost full.
- Agreeing with Student 4 on how a process asks for memory and gives it back.

How did I solve those problems?

- The PID counter only goes up and is never reused.
- The shell checks that the PID is a whole number, and the ProcessManager returns 'Error: Process does not exist.' for an unknown PID.
- I wrote kill_all and it runs on exit and on logout.
- run asks the MemoryManager for 16 MB first. If the answer is no, the process is not created and an error is shown.
- We agreed on two methods, reserve_for_process and release_from_process, so the ProcessManager never changes the memory counters directly.

What did I learn from this project?

I learned that a process is more than a name: it has an identity (the PID), a state and resources. Building the process table showed me how an operating system keeps track of everything that is running, and working with Student 4 showed me why starting and stopping a program has to be linked to memory accounting.

Evidence of my contribution:

- processes.py (Process and ProcessManager)
- The commands process, run, kill, suspend and resume in main.py

Sample output from my part of the program:

```
MiniOS \> run calculator

Process started.

PID: 1
Name: calculator
Status: RUNNING

MiniOS \> process
PID    NAME           STATUS     MEM
1      calculator     RUNNING    16 MB

MiniOS \> kill 999
Error: Process does not exist.
```

Screenshots (attach in the screenshots folder and list the file names here):